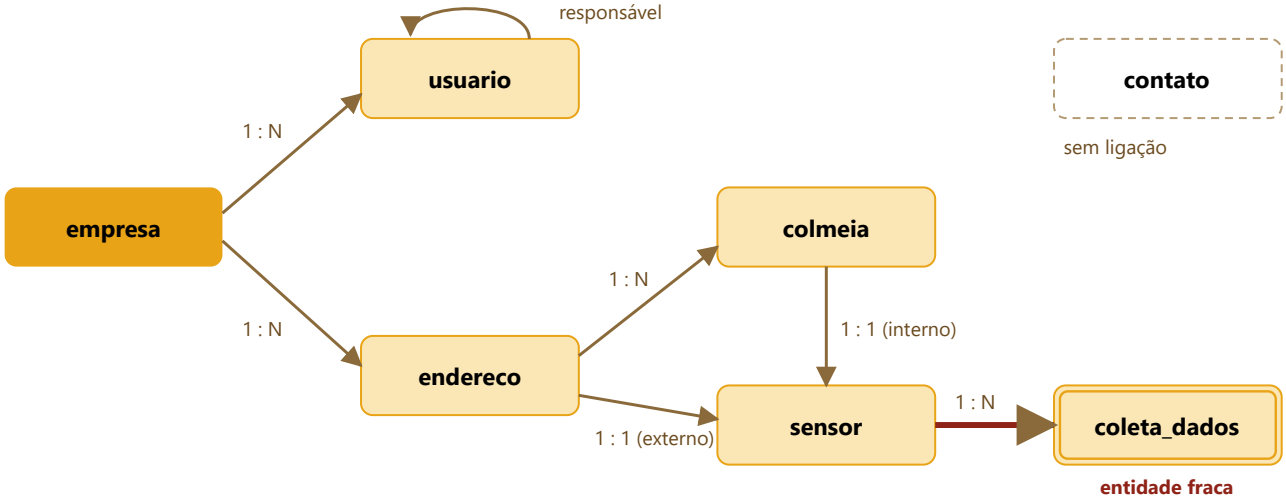
# Banco de Dados Tetria

O que o script cria, como as tabelas se ligam e o que cada consulta responde

## 1. Para que serve

O banco guarda tudo o que o sistema Tetria precisa para monitorar colmeias: as **fazendas** clientes, seus **endereços**, as **colmeias** de cada endereço, os **sensores** instalados e as **leituras** de temperatura e umidade que eles enviam. Também guarda os **usuários** que acessam o sistema e as mensagens do formulário de **contato** do site.

## 2. Como as tabelas se ligam




Lendo o desenho: uma **empresa** tem vários usuários e vários endereços. Cada **endereço** tem várias colmeias e um sensor externo. Cada **colmeia** tem um sensor interno. Cada **sensor** gera várias leituras, e essa é a ligação fraca do modelo (linha vermelha). A tabela **contato** fica sozinha, porque guarda mensagens de quem ainda não é cliente.

> **Por que essa ordem no script?** Uma tabela só pode apontar para outra que já existe. Por isso o script cria e preenche na ordem: empresa → usuario → endereco → colmeia → sensor → coleta_dados.

## 3. Vocabulário rápido

| Comando | Significa |
|---|---|
| `PRIMARY KEY` | O "RG" da linha: único e obrigatório. |
| `AUTO_INCREMENT` | O banco numera sozinho: 1, 2, 3... |
| `FOREIGN KEY` | Liga a linha a outra tabela. Impede, por exemplo, uma colmeia num endereço que não existe. |
| `NOT NULL` / `UNIQUE` | Campo obrigatório / valor que não pode se repetir. |
| `DEFAULT` | Valor usado quando nada é informado (ex.: data atual, status 'Ativo'). |

| | |
|---|---|
| `CHECK` | Regra que o valor precisa cumprir para ser aceito. |

## 4. As tabelas

### `empresa`

A fazenda cliente.

- A chave é um código de texto, `codempresa` (ex.: EMP00001), e não um número.
- `cnpj`, `email` e `telefone` não podem se repetir.

### `usuario`

Quem faz login no sistema.

- Pertence a uma empresa (`fkCodEmpresa`).
- `fkresponsavel` aponta para **outro usuário**, o gestor. Quem é gestor fica com `NULL`.

### `endereco`

Onde ficam os apiários. Uma fazenda pode ter vários.

- Ligado à empresa por `fkEnderecoEmpresa`.

### `colmeia`

Cada caixa de abelhas monitorada.

- `statuss`: 1 = ativa, 0 = inativa. O `CHECK` só aceita esses dois valores.

### `sensor`

O aparelho DHT11 que mede temperatura e umidade.

- `StatusSensor`: 'Ativo' ou 'Inativo'.
- Pode ser **interno** ou **externo** (veja abaixo).

### `coleta_dados`

Cada leitura enviada por um sensor. É a **entidade fraca** do modelo.

- A chave primária é composta: `(idcoleta, fksensor)`.
- `DECIMAL(10,2)` guarda valores com duas casas, como 35.50.
- `dt_hora` registra o momento da leitura.

### `contato`

Mensagens do formulário "Contato" do site. Todos os campos são obrigatórios, e o `CHECK` exige que o e-mail tenha um `@`.

### A regra mais esperta: sensor interno × externo

O sensor **interno** fica dentro de uma colmeia. O **externo** mede o clima do local. Um `CHECK` garante que cada sensor aponte só para o lugar certo:

```
(tiposensor = 'Interno' AND fkcolmeia IS NOT NULL AND fkEndereco_sExterno IS NULL)
OR
(tiposensor = 'Externo' AND fkcolmeia IS NULL AND fkEndereco_sExterno IS NOT NULL)
```

Ou seja: o interno tem colmeia e não tem endereço; o externo tem endereço e não tem colmeia. Como `fkcolmeia` e `fkEndereco_sExterno` são `UNIQUE`, cada colmeia tem **no máximo um** sensor interno e cada endereço tem **no máximo um** sensor externo.

### O relacionamento fraco: sensor → coleta_dados

Uma entidade é **fraca** quando não existe sozinha: ela depende de outra para ser identificada. Uma leitura de 35 °C não quer dizer nada se não soubermos **qual sensor** a fez. Por isso a chave estrangeira `fksensor` também faz parte da chave primária:

```
idcoleta INT NOT NULL AUTO_INCREMENT,
fksensor INT NOT NULL,
PRIMARY KEY (idcoleta, fksensor),
FOREIGN KEY (fksensor) REFERENCES sensor(idsensor)
```

Assim, a leitura é identificada pelo par "número da coleta + sensor". No MySQL Workbench, esse tipo de ligação aparece como **linha contínua** (relação identificadora), e o `fkSensor` aparece com a **chave vermelha**. As outras ligações são tracejadas, porque nelas a chave estrangeira não faz parte da chave primária.

## 5. Dados de exemplo

| Tabela | Linhas | Destaque |
|---|---|---|
| empresa | 5 | Fazendas e apiários de SP, PR, RJ e MG |
| endereco | 7 | Vale Dourado e Santa Helena têm dois endereços cada |
| usuario | 10 | Dois por empresa: um gestor e um funcionário subordinado a ele |
| colmeia | 10 | COL004, COL007 e COL010 estão inativas |
| sensor | 15 | 10 internos (um por colmeia) e 5 externos (um em cada endereço 1, 3, 5, 6 e 7), dos quais 2 estão inativos |
| coleta_dados | 10 | Uma leitura por sensor interno, montada para dar 5 ideais, 2 em atenção e 3 em risco |
| contato | 5 | Mensagens de exemplo de suporte, comercial, financeiro etc. |

## 6. Consultas de exercício

| # | Pergunta | Recurso usado |
|---|---|---|
| 1 | Usuários em ordem alfabética | `ORDER BY` |
| 2 | Colmeias inativas | `WHERE` |
| 3 | Tipos de sensor, sem repetir | `DISTINCT` |
| 4 | Usuário e a fazenda dele | `JOIN` entre duas tabelas |
| 5 | Fazendas e endereços, até as sem endereço | `LEFT JOIN`: mantém a fazenda mesmo sem par |
| 6 | Colmeias inativas e a fazenda delas | Dois `JOIN`s: colmeia → endereço → empresa |
| 7 | Código e local numa coluna só | `CONCAT` |
| 8 | Colmeia "Ativa" ou "Inativa" por extenso | `CASE`: transforma 1/0 em texto |
| 9 | Quantas colmeias cada fazenda tem | `LEFT JOIN` + `COUNT` + `GROUP BY` |
| 10 | Fazenda, colmeia e sensor interno | Três `JOIN`s + filtro por tipo |

A consulta de **contato** usa `CONCAT` para montar uma frase por mensagem, e `DATE_FORMAT` para mostrar a data no formato brasileiro (25/10/2026 14:30).

## 7. Consultas do dashboard

Elas alimentam a tela de monitoramento. Todas usam a mesma régua para classificar uma colmeia a partir da leitura do sensor interno:

| Situação | Temperatura | | Umidade |
|---|---|---|---|
| Risco | abaixo de 32 °C **ou** acima de 38 °C | **ou** | 85% ou mais |
| Ideal | entre 33 e 36 °C | **e** | entre 50% e 75% |
| Atenção | qualquer outro caso (ex.: 32 °C, ou umidade de 80%) | | |

> **A ordem do `CASE` importa.** O banco testa de cima para baixo e para no primeiro que der certo. Por isso o Risco vem primeiro: uma colmeia a 35 °C com 90% de umidade cai em Risco antes de chegar ao teste de Ideal.

| Consulta | O que mostra | Resultado com os dados de exemplo |
|---|---|---|
| Ação e informações | Cada colmeia com sensor interno ativo, a leitura e a situação | 10 linhas |
| Quantidade por estado | Quantas colmeias há em cada situação. Uma subconsulta classifica cada colmeia, e o `GROUP BY` conta | 5 2 3 |
| Colmeias em risco | Só as que precisam de ação. Os parênteses no `WHERE` agrupam as três condições de risco | COL002, COL008 e COL010 |
| Sensores ativos e inativos | Total de sensores por status | 13 ativos, 2 inativos |
| Sensores inativos | Quais estão desligados | Sensores 6 e 12 (externos) |

No fim do script, `DROP DATABASE tetria;` fica comentado. Basta tirar o `--` para apagar o banco e rodar tudo de novo do zero.